Introduction to Verilog Hardware Description Language (HDL)

Engr. Mario Jr. G. Brucal
Guest Faculty
College of Engineering
Laguna State Polytechnic University
San Pablo City Campus

February 07, 2026

Learning Objectives

- Explain the design flow.
- Differentiate between Lexical Conventions and Data Types in Verilog.

Introduction to Verilog HDL

Verilog is a **Hardware Description Language (HDL)** used extensively to design and simulate digital systems, ranging from simple combinational logic to complex microprocessors. Originally developed by Gateway Design Automation in 1984 and later standardized as IEEE 1364, **Verilog allows designers to describe hardware at multiple levels of abstraction, including behavioral, dataflow, and structural levels**.

Introduction to Verilog HDL

This lecture addresses two primary objectives:

1. **The Design Flow**: Understanding how we move from an abstract idea to a physical chip.

2. Language Fundamentals: Differentiating between the lexical rules (syntax) and the data types (semantics) used to model hardware.

The Design Flow

The design of digital systems using Verilog follows a structured methodology, often referred to as a **Top-Down Design Flow**. This process moves from high-level abstract specifications to low-level physical implementation.

The Design Flow

1. Specification and Behavioral Description

The flow begins with a design specification, which describes the functionality, interface, and architecture of the digital circuit. Designers often create a behavioral description first. This is a high-level algorithmic description (similar to C programming) that defines what the circuit does without detailing how it is implemented in hardware.

The Design Flow

2. Register Transfer Level (RTL) Coding

The most critical phase for a Verilog designer is converting the behavioral description into RTL (Register Transfer Level) code. RTL specifies the flow of data between registers and how that data is processed. This description is technology-independent, meaning it defines functionality without tying the design to a specific manufacturer's silicon.

The Design Flow

3. Functional Verification (Simulation)

Before proceeding, the RTL must be verified. Functional verification involves applying stimulus (input signals) to the design via a test bench to ensure the output matches the specification. A test bench is a Verilog module that generates signals to drive the design under test (DUT) and monitors the response.

The Design Flow

4. Logic Synthesis

Once functionality is verified, logic synthesis tools convert the RTL description into a gate-level netlist. Synthesis is analogous to compiling code in software; it translates abstract Verilog constructs (like if-else statements or arithmetic operators) into specific logic gates (AND, OR, Flip-Flops) available in a target technology library.

The Design Flow

5. Place and Route (Physical Design)

The gate-level netlist is input into an automatic "Place and Route" tool. This step determines the physical location of each gate on the silicon die and wires them together.

The Design Flow

6. Timing Verification

After the physical layout, the design undergoes timing simulation. Real hardware has delays; this step ensures that signals arrive at their destinations within the required timeframes (setup and hold times) to prevent failure.

Lexical Conventions vs Data Types

To write valid Verilog, one must distinguish between Lexical Conventions (the grammatical rules of the text) and Data Types (the hardware abstractions representing information).

A. Lexical Conventions

Verilog source files are streams of tokens. The layout is free-format, meaning spaces and newlines are generally ignored except when separating tokens.

Lexical Conventions vs Data Types

A. Lexical Conventions

1. Whitespace and Comments. Whitespace includes blanks, tabs, and newlines. Comments are used for documentation and are ignored by the compiler.

- **Single-line comments start with // and end at the newline.**
- **Multi-line comments are enclosed between /* and */.**

Lexical Conventions vs Data Types

A. Lexical Conventions

2. Identifiers and Keywords

- Identifiers are names given to objects (modules, signals, variables). They must start with an alphabetic character or an underscore (_) and can contain alphanumeric characters and the dollar sign ($). They are case-sensitive; **Data** and **data** are **different identifiers**.
- Keywords are reserved words defining language constructs (e.g., module, reg, wire, input). **All keywords must be in lowercase.**

Lexical Conventions vs Data Types

A. Lexical Conventions

3. Number Specification. Numbers in Verilog can be sized or unsized.

- **Format**: <size>'<base_format><number>
- **Size**: Decimal number specifying the number of bits.
- **Base Format**: **b (binary), d (decimal), h (hexadecimal), o (octal).**

Lexical Conventions vs Data Types

A. Lexical Conventions

4. Operators. Verilog uses single, double, or triple character sequences for operations. Examples include unary (e.g., ~ for NOT), binary (e.g., && for logical AND), and ternary (e.g., ?: for conditional) operators.

Lexical Conventions vs Data Types

B. Data Types

Data types determine how values are stored and transmitted in hardware. Verilog has a four-value logic set:

- 0: Logic zero (False)
- 1: Logic one (True)
- x: Unknown logic value
- z: High impedance (floating state).

Verilog data types are broadly divided into **Nets** and **Registers (Variables).**

Lexical Conventions vs Data Types

B. Data Types

1. **Nets (Physical Connections).** Nets represent physical connections between structural entities, such as logic gates. **They do not store values; they continuously reflect the value of their driving source.**

Keyword: The most common net type is wire. Others include tri, wand, and supply0.

Usage Rule: Nets are driven by the output of a module, a primitive gate, or a continuous assignment (using the assign keyword).

Default Value: If unconnected, a net usually defaults to z (high impedance).

Lexical Conventions vs Data Types

B. Data Types

2. Registers/Variables (Storage). Variables represent data storage elements. They retain their value until a new value is assigned to them, similar to variables in software programming.

Keyword: The most common variable type is reg. Note that reg does not necessarily imply a physical hardware register (flip-flop); it simply means the variable holds a state in the simulation.

Usage Rule: Variables are assigned values inside procedural blocks (such as always or initial blocks).

Default Value: Variables of type reg initialize to x (unknown) at the start of simulation.

Lexical Conventions vs Data Types

B. Data Types

3. Vectors vs. Scalars

- **Scalar:** A 1-bit net or register (e.g., wire a;).
- **Vector:** A multi-bit net or register, declared using a range [MSB:LSB].
    - **Example: reg [7:0]** bus; defines an 8-bit bus where bit 7 is the Most Significant Bit (MSB).
    - **Part Select:** You can access specific bits, such as bus[2:0].

Lexical Conventions vs Data Types

B. Data Types

4. Other Data Types

- **Integer:** General-purpose variables (usually 32-bit signed) used for loop counting and calculation, not usually for hardware signals.
- **Time:** A 64-bit unsigned variable used to store simulation time, typically used with the $time system function.
- **Real:** Supports floating-point numbers for calculation (e.g., real delta = 2.13;).
- **Parameters:** These represent constants, not variables. They allow module instances to be customized (e.g., defining a bus width) at compile time.

Thank you!

Engr. Mario Jr. G. Brucal
Guest Faculty
College of Engineering
Laguna State Polytechnic University
San Pablo City Campus

February 07, 2026